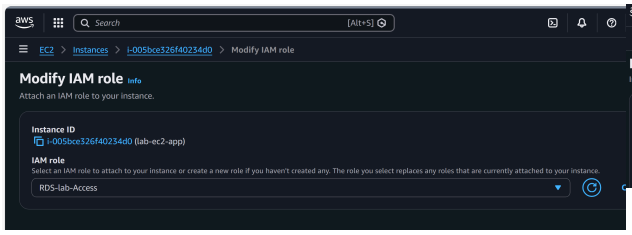
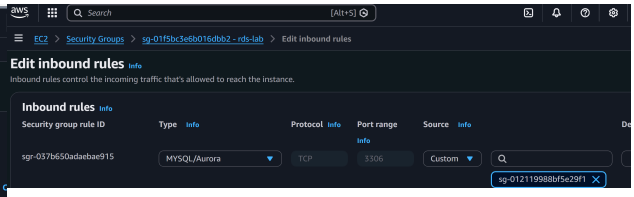
Lab 1A Deliverables

Student Deliverables:
1) Screenshot of:
  RDS SG inbound rule using source = sg-ec2-lab
  EC2 role attached
  /list output showing at least 3 notes

2) Short answers:
  A) Why is DB inbound source restricted to the EC2 security group?
  The answer in my own words would be: Because we don't want anybody getting access to this database, and by it only being accessed by the EC2, that will be the only thing that can write into it. Otherwise, other people will be able to put stuff into our database or even launch attacks, and it will leave the database vulnerable.

  B) What port does MySQL use?
  The Port is 3306.
  C) Why is Secrets Manager better than storing creds in code/user-data?
  In my own words, I would say the Secrets Manager is better because it is natively in AWS. Another unique benefit of using the Secrets Manager is that you don't have to worry about risking the credentials on your actual device and it's inside of your account, and you don't have to go fiddle around with passwords. So the risk of loss is lowered. And then with it also being within AWS, it has a feature where you could even rotate the keys automatically. That way you don't have to worry about remembering to change the password every single time.





A:) Notes

A) Why is DB inbound source restricted to the EC2 security group?

The answer in my own words would be: Because we don't want anybody getting access to this database, and by it only being accessed by the EC2, that will be the only thing that can write into it. Otherwise, other people will be able to put stuff into our database or even launch attacks, and it will leave the database vulnerable.

Your answer is **on point** and hits the core concept! You're correctly identifying the **principle of least privilege** — only the resources that *need* access should have it. Here's a quick breakdown:

✅ What you nailed:

- The DB should only be accessible by the EC2 instance (not the public internet or random sources)
- Restricting access prevents unauthorized writes and potential attacks
- You captured the "why" behind the security posture

🔥 Flavor text to level it up:

> **"By setting the RDS security group inbound rule to only allow traffic from the EC2 security group (sg-ec2-lab), we're implementing the principle of least privilege. This ensures that:**

> 1. **Only trusted compute resources** (in this case, our application on the EC2 instance) can initiate connections to the database on port 3306.

> 2. **The database has no direct public exposure**, reducing the attack surface.

> 3. **Security group referencing** (using sg-id instead of IP ranges) means if the EC2's IP changes, the rule still works — it's dynamic and maintainable.

> 4. **Defense in depth** — even if someone compromises another resource in the VPC, they can't reach the DB unless they're part of that specific EC2 security group."

💡 Bonus term to drop:

This is also an example of **"security group chaining"** — where one security group references another, creating a trust relationship between tiers (app tier → data tier).

You're good to go — solid understanding of VPC security fundamentals! 🎯

C notes)

Why is Secrets Manager better than storing creds in code/user-data?
In my own words, I would say the Secrets Manager is better because it is natively in AWS. Another unique benefit of using the Secrets Manager is that you don't have to worry about risking the credentials on your actual device and it's inside of your account, and you don't have to go fiddle around with passwords. So the risk of loss is lowered. And then with it also being within AWS, it has a feature where you could even rotate the keys automatically. That way you don't have to worry about remembering to change the password every single time.

Your answer is **solid** and captures several key benefits! You're hitting on important security principles. Let me break it down:

✅ What you nailed:

- **Native AWS integration** — credentials stay within the AWS ecosystem
- **Automatic rotation** — huge security win, reduces human error
- **Reduced risk of exposure** — not hardcoded on your machine or in plaintext

⚠️ One small clarification:

When you say "inside your account," that's true — but the *real* magic is that **Secrets Manager encrypts secrets at rest using KMS**, so even if someone got into your account, they'd still need KMS permissions to decrypt them. That's a layer you could add.

🔥 Flavor text to level it up:

> **"Secrets Manager is superior to hardcoding credentials in code or user-data because:**

> 1. **No plaintext exposure** — Credentials in code/user-data can be leaked via version control (Git), logs, or instance metadata. Secrets Manager stores them encrypted at rest with AWS KMS.

> 2. **Automatic rotation** — Secrets Manager can rotate database credentials on a schedule (e.g., every 30 days) without manual intervention or app downtime.

> 3. **IAM-based access control** — Only resources with the right IAM policies can retrieve the secret. Your EC2 instance uses its attached role to authenticate — no credentials baked into the AMI or user-data script.

> 4. **Audit trail** — Every secret access is logged in CloudTrail, giving you full visibility into who accessed what and when.

> 5. **No credential sprawl** — One secret, one source of truth. You don't have copies floating around in config files, scripts, or emails."

💡 Bonus point to drop:

Hardcoded creds in **user-data** are especially risky because user-data is accessible via the **instance metadata service (IMDS)** at `169.254.169.254`. If an

attacker gets SSRF or shell access, they can read it directly. Secrets Manager + IAM roles eliminates that vector entirely.

You're crushing these — keep it up! 💪